

Mini-projet — Construire un crawler avec Scrapy

Contexte

Vous venez de découvrir le fonctionnement de Scrapy.

Votre objectif est maintenant de construire un **crawler complet** capable de parcourir automatiquement un site web, de suivre plusieurs niveaux de pages et de produire un jeu de données structuré.

Vous travaillerez sur le site d'entraînement :

<https://books.toscrape.com/>

Ce site contient environ **1000 livres**, répartis sur plusieurs pages.

Chaque livre possède également une page de détail contenant des informations supplémentaires.

Objectif

Créer un projet **Scrapy** permettant de récupérer automatiquement les informations disponibles sur l'ensemble du catalogue de `books.toscrape.com`.

Votre spider devra :

1. parcourir automatiquement toutes les pages du catalogue ;
2. identifier tous les livres présents ;
3. suivre la page de détail de chaque livre ;
4. extraire les informations demandées ;
5. construire des données structurées ;
6. exporter les résultats dans un fichier CSV ou JSON.

Aucune liste d'URL ne doit être écrite manuellement.

1. Création du projet

Créez un nouveau projet Scrapy nommé :

`books_scraper`

Votre projet devra respecter l'organisation classique d'un projet Scrapy.

Exemple attendu :

```
books_scraper/
├── scrapy.cfg
└── books_scraper/
    ├── __init__.py
    ├── items.py
    ├── middlewares.py
    ├── pipelines.py
    ├── settings.py
    └── spiders/
        ├── __init__.py
        └── books.py
```

2. Spider principal

Créez un spider nommé :

books

Le point de départ du crawler sera :

<https://books.toscrape.com/>

Le spider devra automatiquement détecter les livres présents sur la page.

Vous devrez utiliser les **sélecteurs CSS ou XPath de Scrapy**.

3. Informations à récupérer

Pour chaque livre, récupérez les données suivantes.

Informations disponibles dans le catalogue

Champ	Description
<code>title</code>	titre du livre
<code>price</code>	prix du livre
<code>star_rating</code>	note entre 1 et 5

Champ	Description
<code>in_stock</code>	disponibilité du livre
<code>thumbnail_url</code>	URL de la miniature
<code>detail_url</code>	URL de la page du livre

Exemple de donnée intermédiaire :

```
{
  "title": "A Light in the Attic",
  "price": 51.77,
  "star_rating": 3,
  "in_stock": True
}
```

4. Suivre la page de détail

Certaines informations ne sont pas présentes directement dans le catalogue.

Votre spider doit donc suivre le lien de chaque livre avec Scrapy.

Vous pourrez notamment utiliser :

```
response.follow(...)
```

ou

```
yield response.follow(...)
```

Sur la page de détail, récupérez :

Champ	Description
<code>upc</code>	identifiant UPC
<code>description</code>	description du livre
<code>number_available</code>	nombre d'exemplaires disponibles
<code>category</code>	catégorie du livre
<code>image_url</code>	URL de l'image principale

Le résultat final doit regrouper les informations provenant du catalogue **et** celles provenant de la page de détail.

5. Pagination

La première page ne contient qu'une partie des livres.

Votre spider doit trouver automatiquement le bouton :

```
next
```

puis suivre cette URL.

Le programme doit continuer jusqu'à ce qu'il n'existe plus de page suivante.

Vous ne devez donc pas écrire :

```
for page in range(1, 51):
```

Le crawler doit découvrir la pagination directement à partir du HTML.

6. Scrapy Item

Créez un `Item` représentant un livre dans :

```
items.py
```

Il devra au minimum contenir les champs suivants :

```
title
price
star_rating
in_stock
thumbnail_url
detail_url
upc
description
number_available
category
image_url
```

Votre spider devra retourner cet Item une fois toutes les informations récupérées.

7. Nettoyage des données

Les données récupérées doivent être propres et exploitables.

Par exemple :

£51.77

doit devenir :

51.77

Le stock :

In stock (22 available)

doit permettre d'obtenir :

22

La note :

<p class="star-rating Three">

doit permettre d'obtenir :

3

Les URLs des images doivent être des **URLs absolues** et non des chemins relatifs.

8. Export

Votre spider doit pouvoir produire un fichier :

books.csv

ou :

```
books.json
```

Exemple :

```
scrapy crawl books -o books.csv
```

Le fichier doit contenir une ligne par livre.

Résultat attendu

À la fin de l'exécution, votre crawler doit avoir parcouru l'intégralité du catalogue.

Le fichier final devra ressembler à :

```
title,price,star_rating,in_stock,upc,category,number_available
A Light in the Attic,51.77,3,True,a897fe39b1053632,Poetry,22
Tipping the Velvet,53.74,1,True,90fa61229261140a,Historical Fiction,20
Soumission,50.10,1,True,6957f44c3847a760,Fiction,20
```

Contraintes

Votre projet doit obligatoirement :

- utiliser **Scrapy** ;
 - utiliser un `Spider` ;
 - utiliser les selectors Scrapy ;
 - utiliser `response.follow()` ou une logique équivalente ;
 - gérer automatiquement la pagination ;
 - visiter les pages de détail ;
 - utiliser un `Item` ;
 - produire un CSV ou JSON ;
 - éviter les URLs écrites manuellement pour chaque page ;
 - éviter `requests` et `BeautifulSoup` .
-

Organisation conseillée

Travaillez progressivement.

Étape 1 — Premier scraping

Commencez par récupérer uniquement :

```
title  
price
```

sur la première page.

Vérifiez que vous récupérez bien les **20 livres** présents.

Étape 2 — Item

Créez votre `BookItem` et retournez vos résultats sous cette forme.

Étape 3 — Informations du catalogue

Ajoutez :

```
star_rating  
in_stock  
thumbnail_url
```

Étape 4 — Page de détail

Pour chaque livre :

```
Catalogue  
|  
| response.follow()  
▼  
Page du livre
```

Récupérez :

```
UPC  
Description  
Stock exact  
Catégorie  
Image
```

Étape 5 — Pagination

Ajoutez la navigation automatique :

```
Page 1
  ↓
Page 2
  ↓
Page 3
  ↓
...
  ↓
Dernière page
```

Étape 6 — Export

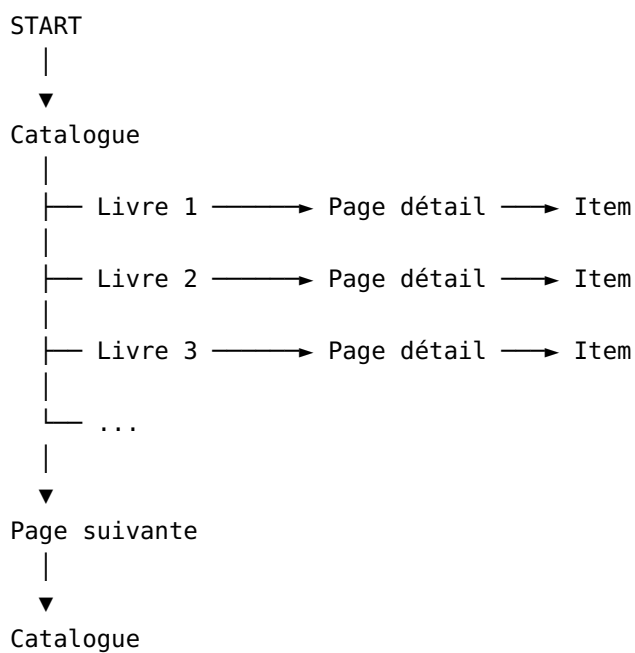
Exportez votre dataset.

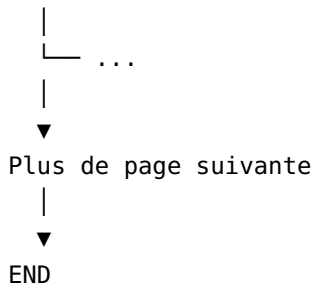
```
scrapy crawl books -o books.csv
```

Puis vérifiez le contenu du fichier.

Structure logique attendue

Votre crawler devra approximativement suivre cette logique :

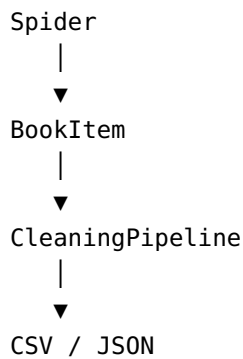




● Bonus 1 — Pipeline

Créez un `ItemPipeline` chargé de nettoyer les données.

Par exemple :



Le pipeline pourra notamment :

- convertir le prix en `float` ;
- convertir le stock en `int` ;
- convertir la note en `int` ;
- supprimer les espaces inutiles dans les descriptions.

● Bonus 2 — Téléchargement des images

Utilisez le système de pipeline Scrapy pour télécharger automatiquement les images des livres.

Les images devront être stockées localement dans un dossier dédié.

Bonus 3 — Statistiques

Une fois le scraping terminé, utilisez votre fichier pour répondre aux questions suivantes :

- Combien de livres ont été récupérés ?
- Quel est le prix moyen des livres ?
- Quel est le livre le plus cher ?
- Quel est le livre le moins cher ?
- Quelle catégorie contient le plus de livres ?
- Quelle est la note moyenne des livres ?
- Combien de livres ont une note de 5 étoiles ?

Challenge

Ajoutez une option permettant de lancer uniquement le scraping d'une catégorie.

Par exemple :

```
scrapy crawl books -a category=travel
```

Le crawler ne devra alors récupérer que les livres de la catégorie demandée.

Livrable

Vous devez rendre un repository GitHub contenant au minimum :

```
books_scraper/  
├─ scrapy.cfg  
├─ books_scraper/  
│   ├── items.py  
│   ├── pipelines.py  
│   ├── settings.py  
│   └── spiders/  
│       └─ books.py  
├─ README.md  
├─ requirements.txt  
└─ books.csv
```

Le `README.md` devra expliquer :

- l'objectif du projet ;
- comment installer les dépendances ;

- comment lancer le spider ;
- comment exporter les données ;
- la structure générale du crawler.

Commandes attendues dans le README

Installation :

```
pip install -r requirements.txt
```

Lancement :

```
scrapy crawl books
```

Export CSV :

```
scrapy crawl books -o books.csv
```

Export JSON :

```
scrapy crawl books -o books.json
```

Critères de réussite

Le projet est considéré comme fonctionnel si :

- le spider démarre correctement ;
- les livres sont détectés ;
- les pages de détail sont visitées ;
- la pagination fonctionne automatiquement ;
- les données sont correctement nettoyées ;
- environ 1000 livres sont récupérés ;
- le CSV/JSON est correctement généré ;
- aucune URL des 50 pages n'est écrite manuellement.